

8.5 Volatility Forecasting in South Africa ZA

In this assignment you'll build a model to predict stock volatility for the telecommunications company [MTN Group](#).

Tip: There are some tasks in this assignment that you can complete by importing functions and classes you created for your app. Give it a try!

Warning: There are some tasks in this assignment where there is an extra code block that will transform your work into a `submission` that's compatible with the grader. Be sure to run those cells and inspect the `submission` before you submit to the grader.

```
In [1]: %load_ext autoreload
        %autoreload 2

        from arch.univariate.base import ARCHModelResult
```

Note about Alpha Vantage API access

In this lesson, we will mock the requests library instead of making real requests to the Alpha Vantage API.

Some students have had issues with the live API, especially when requesting the full output size or reaching the daily request limit. Mocking the response allows everyone to work with the same expected data and focus on the lesson content.

Please note that if you open the API link directly in your web browser, you may still see the original API issue. This is expected, because the browser request is not mocked by our local lesson infrastructure.

```
In [2]: from mock_alpha import activate_mock
        activate_mock()
```

```
In [3]: import sqlite3
        import matplotlib.pyplot as plt
        import numpy as np
        import pandas as pd
        import requests
        from arch import arch_model
        from config import settings
        from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

Working with APIs

Task 8.5.1

```
In [4]: ticker = "MTNOY"
output_size = "full"
data_type = "json"

url = (
    "https://learn-api.wqu.edu/1/data-services/alpha-vantage/query?"
    "function=TIME_SERIES_DAILY&"
    f"symbol={ticker}&"
    f"outputsize={output_size}&"
    f"datatype={data_type}&"
    f"apikey={settings.alpha_api_key}"
)

print("url type:", type(url))
url
```

url type: <class 'str'>

```
Out[4]: 'https://learn-api.wqu.edu/1/data-services/alpha-vantage/query?function=TIME_SE
RIES_DAILY&symbol=MTNOY&outputsize=full&datatype=json&apikey=...'
```

Task 8.5.2

```
In [5]: response = requests.get(url=url)
print("response type:", type(response))
```

response type: <class 'unittest.mock.Mock'>

Task 8.5.3

```
In [6]: response_code = response.status_code
print("code type:", type(response_code))
response_code
```

code type: <class 'int'>

```
Out[6]: 200
```

Test-Driven Development

Task 8.5.4

```
In [7]: # Extract the time series data
response_data = response.json()
stock_data = response_data["Time Series (Daily)"]

# Build DataFrame
df_mtnoy = pd.DataFrame.from_dict(stock_data, orient="index", dtype=float)

# Convert index to DatetimeIndex
df_mtnoy.index = pd.to_datetime(df_mtnoy.index)
df_mtnoy.index.name = "date"
```

```
# Clean column names (remove the "1. ", "2. ", etc.)
df_mtnoy.columns = [c.split(". ")[1] for c in df_mtnoy.columns]

print("df_mtnoy type:", type(df_mtnoy))
df_mtnoy.head()
```

df_mtnoy type: <class 'pandas.core.frame.DataFrame'>

```
Out[7]:
```

	open	high	low	close	volume
date					
2026-06-04	13.36	13.36	13.26	13.29	6400.0
2026-06-03	13.29	13.32	13.17	13.22	8500.0
2026-06-02	13.19	13.35	13.19	13.32	36400.0
2026-06-01	13.21	13.33	13.17	13.33	5700.0
2026-05-29	13.52	13.52	13.35	13.42	18500.0

Task 8.5.5

```
In [8]: connection = sqlite3.connect(settings.db_name, check_same_thread=False)
connection
```

Out[8]: <sqlite3.Connection at 0x78299e423e20>

Task 8.5.6

```
In [9]: # Insert `MTNOY` data into database
df_mtnoy.to_sql(
    name="MTNOY",
    con=connection,
    if_exists="replace",
    index=True,
)
```

Out[9]: 4119

Task 8.5.7

```
In [10]: df_mtnoy_read = pd.read_sql(
    sql="SELECT * FROM MTNOY",
    con=connection,
    index_col="date",
    parse_dates=["date"],
)

print("df_mtnoy_read type:", type(df_mtnoy_read))
print("df_mtnoy_read shape:", df_mtnoy_read.shape)
df_mtnoy_read.head()
```

df_mtnoy_read type: <class 'pandas.core.frame.DataFrame'>
df_mtnoy_read shape: (4119, 5)

```
Out[10]:
```

	open	high	low	close	volume
date					
2026-06-04	13.36	13.36	13.26	13.29	6400.0
2026-06-03	13.29	13.32	13.17	13.22	8500.0
2026-06-02	13.19	13.35	13.19	13.32	36400.0
2026-06-01	13.21	13.33	13.17	13.33	5700.0
2026-05-29	13.52	13.52	13.35	13.42	18500.0

Predicting Volatility

Prepare Data

Task 8.5.8

```
In [11]: # Sort ascending, compute returns, keep most recent 2500
y_mtnoy = (
    df_mtnoy_read["close"]
    .sort_index(ascending=True)
    .pct_change()
    .dropna()
    .mul(100)
    .rename("return")
    .iloc[-2500:]
)

print("y_mtnoy type:", type(y_mtnoy))
print("y_mtnoy shape:", y_mtnoy.shape)
y_mtnoy.head()
```

```
y_mtnoy type: <class 'pandas.core.series.Series'>
y_mtnoy shape: (2500,)
```

```
Out[11]: date
2016-06-24    -13.216715
2016-06-27     -3.583427
2016-06-28      7.084785
2016-06-29      5.639913
2016-06-30      0.000000
Name: return, dtype: float64
```

Task 8.5.9

```
In [12]: mtnoy_daily_volatility = y_mtnoy.std()
print("mtnoy_daily_volatility type:", type(mtnoy_daily_volatility))
print("MTN Daily Volatility:", mtnoy_daily_volatility)
```

```
mtnoy_daily_volatility type: <class 'numpy.float64'>
MTN Daily Volatility: 2.9229845442145654
```

Task 8.5.10

```
In [13]: mtnoy_annual_volatility = mtnoy_daily_volatility * np.sqrt(252)
print("mtnoy_annual_volatility type:", type(mtnoy_annual_volatility))
print("MTN Annual Volatility:", mtnoy_annual_volatility)
```

mtnoy_annual_volatility type: <class 'numpy.float64'>
MTN Annual Volatility: 46.40094114046333

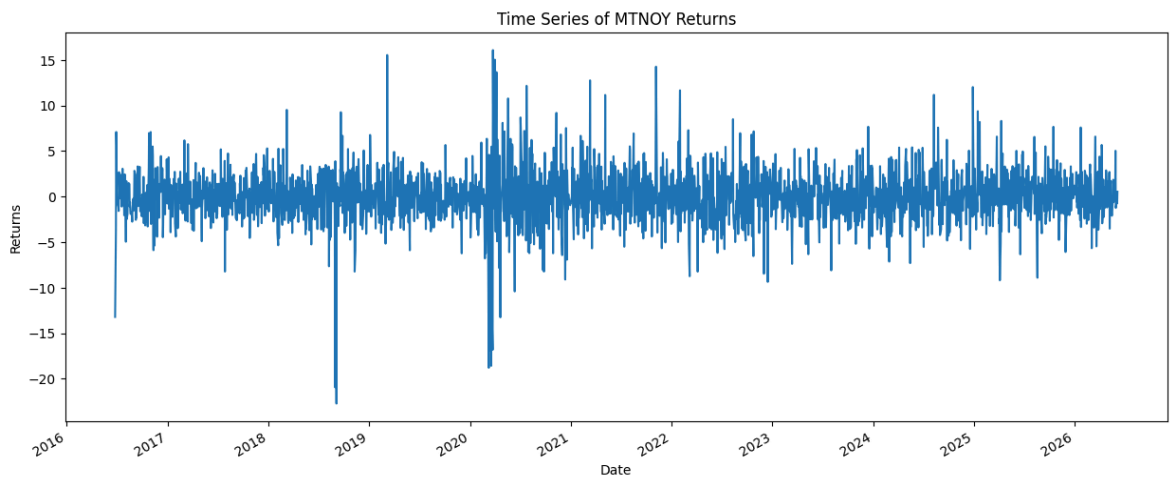
Task 8.5.11

```
In [14]: # Create `fig` and `ax`
fig, ax = plt.subplots(figsize=(15, 6))

# Plot `y_mtnoy` on `ax`
y_mtnoy.plot(ax=ax)

# Add axis labels
ax.set_xlabel("Date")
ax.set_ylabel("Returns")

# Add title
ax.set_title("Time Series of MTNOY Returns");
```



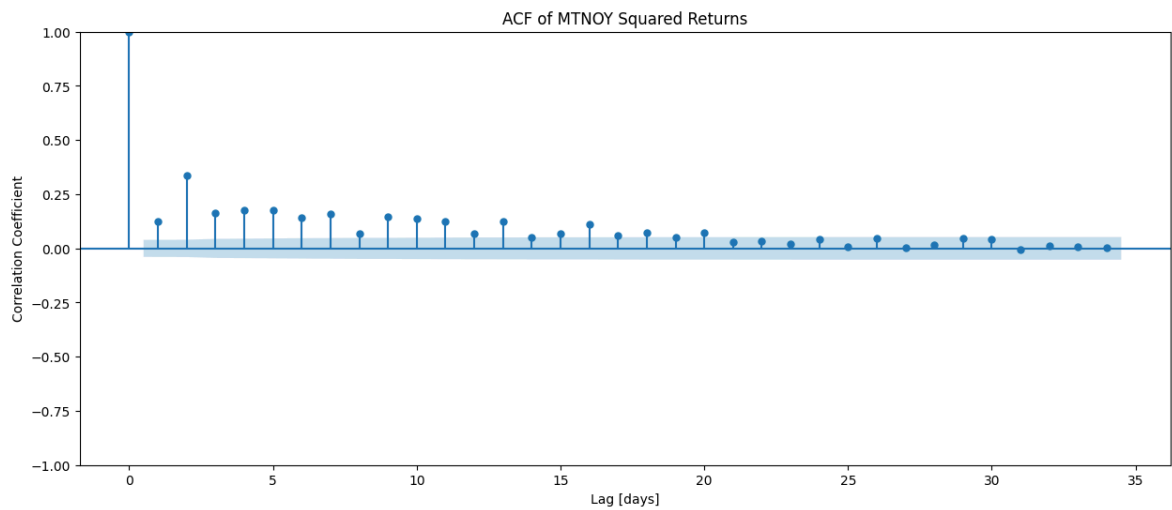
Task 8.5.12

```
In [15]: # Create `fig` and `ax`
fig, ax = plt.subplots(figsize=(15, 6))

# Create ACF of squared returns
plot_acf(y_mtnoy**2, ax=ax)

# Add axis labels
ax.set_xlabel("Lag [days]")
ax.set_ylabel("Correlation Coefficient")

# Add title
ax.set_title("ACF of MTNOY Squared Returns");
```



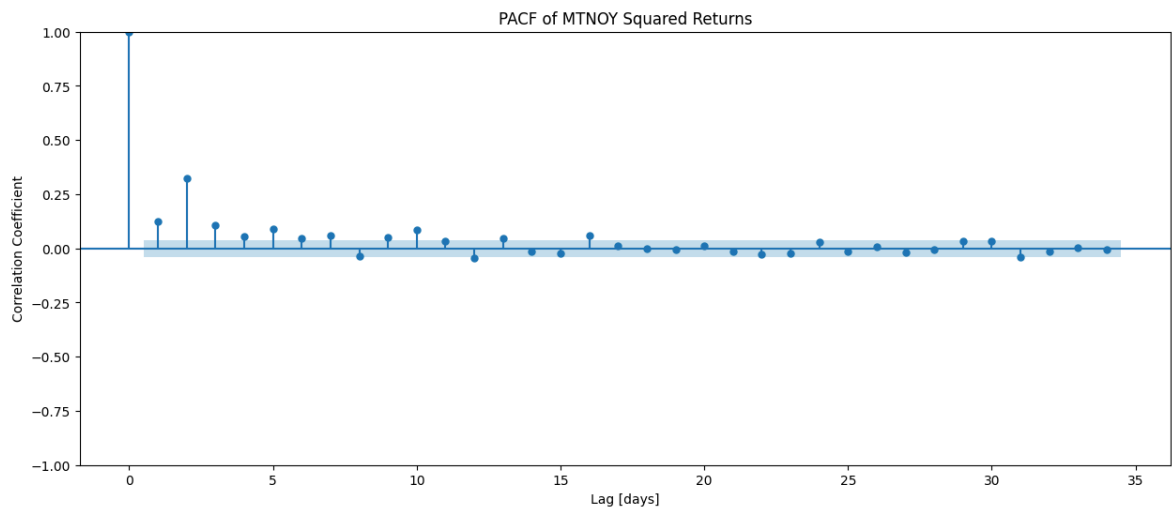
Task 8.5.13

```
In [16]: # Create `fig` and `ax`
fig, ax = plt.subplots(figsize=(15, 6))

# Create PACF of squared returns
plot_pacf(y_mtnoy**2, ax=ax)

# Add axis labels
ax.set_xlabel("Lag [days]")
ax.set_ylabel("Correlation Coefficient")

# Add title
ax.set_title("PACF of MTNOY Squared Returns");
```



Task 8.5.14

```
In [17]: cutoff_test = int(len(y_mtnoy) * 0.8)
y_mtnoy_train = y_mtnoy.iloc[:cutoff_test]

print("y_mtnoy_train type:", type(y_mtnoy_train))
print("y_mtnoy_train shape:", y_mtnoy_train.shape)
y_mtnoy_train.head()

y_mtnoy_train type: <class 'pandas.core.series.Series'>
y_mtnoy_train shape: (2000,)
```

```
Out[17]: date
2016-06-24    -13.216715
2016-06-27     -3.583427
2016-06-28      7.084785
2016-06-29      5.639913
2016-06-30      0.000000
Name: return, dtype: float64
```

Build Model

Task 8.5.15

```
In [18]: # Build and train model
model = arch_model(
    y_mtnoy_train,
    p=1,
    q=1,
    rescale=False,
    vol="GARCH",
    dist="normal",
).fit(dis=0)

print("model type:", type(model))

# Show model summary
model.summary()
```

```
model type: <class 'arch.univariate.base.ARCHModelResult'>
```

Out[18]:

Constant Mean - GARCH Model Results			
Dep. Variable:	return	R-squared:	0.000
Mean Model:	Constant Mean	Adj. R-squared:	0.000
Vol Model:	GARCH	Log-Likelihood:	-4875.92
Distribution:	Normal	AIC:	9759.84
Method:	Maximum Likelihood	BIC:	9782.25
No. Observations:			2000
Date:	Sun, Jul 26 2026	Df Residuals:	1999
Time:	11:22:43	Df Model:	1

Mean Model					
	coef	std err	t	P> t	95.0% Conf. Int.
mu	1.6894e-03	6.010e-02	2.811e-02	0.978	[-0.116, 0.119]

Volatility Model					
	coef	std err	t	P> t	95.0% Conf. Int.
omega	0.3431	0.179	1.914	5.558e-02	[-8.181e-03, 0.694]
alpha[1]	0.0655	2.049e-02	3.197	1.391e-03	[2.533e-02, 0.106]
beta[1]	0.8933	3.825e-02	23.358	1.148e-120	[0.818, 0.968]

Covariance estimator: robust

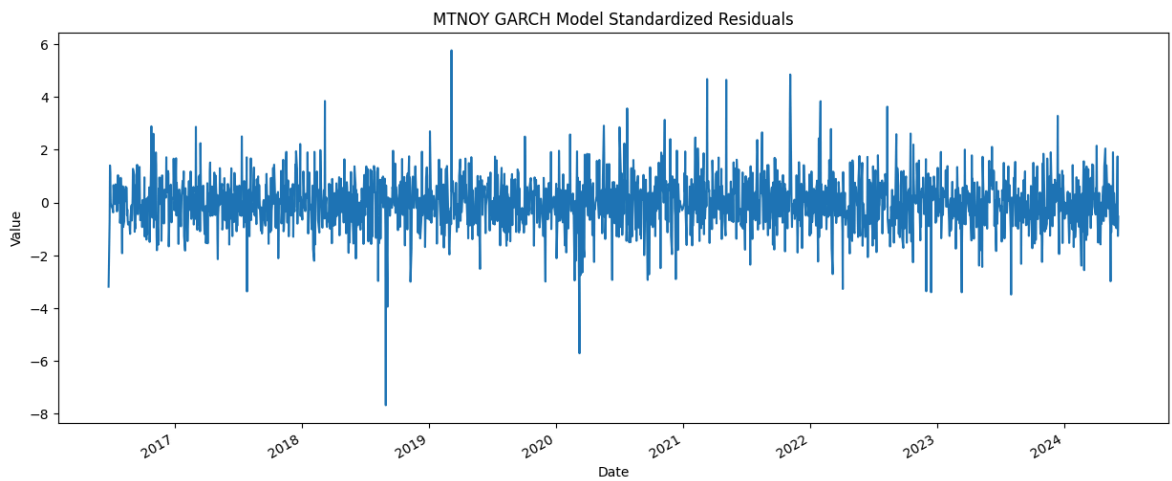
Task 8.5.16

```
In [19]: # Create `fig` and `ax`
fig, ax = plt.subplots(figsize=(15, 6))

# Plot standardized residuals
model.std_resid.plot(ax=ax)

# Add axis labels
ax.set_xlabel("Date")
ax.set_ylabel("Value")

# Add title
ax.set_title("MTNOY GARCH Model Standardized Residuals");
```



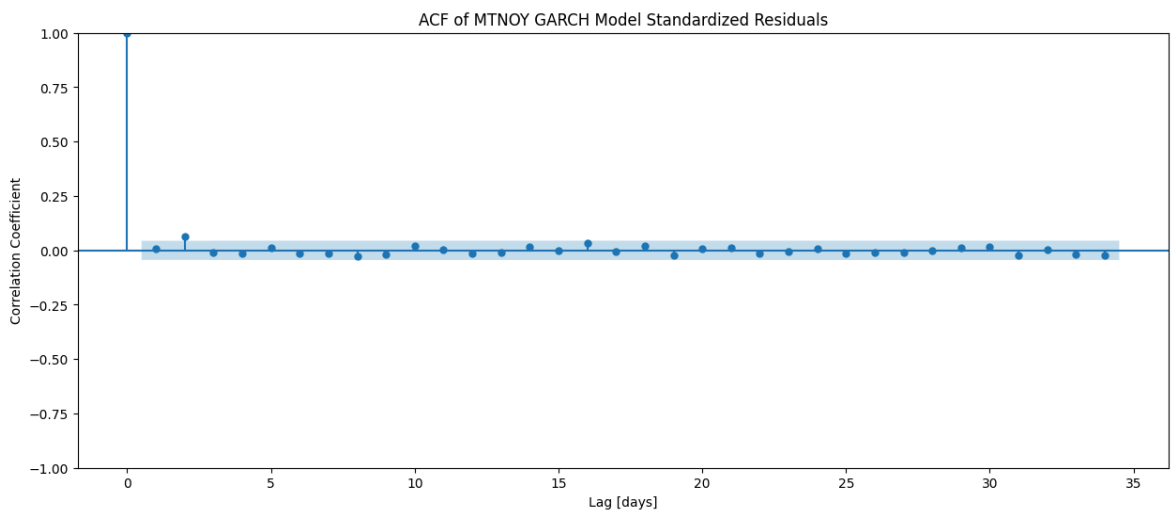
Task 8.5.17

```
In [20]: # Create `fig` and `ax`
fig, ax = plt.subplots(figsize=(15, 6))

# Create ACF of squared, standardized residuals
plot_acf(model.std_resid**2, ax=ax)

# Add axis labels
ax.set_xlabel("Lag [days]")
ax.set_ylabel("Correlation Coefficient")

# Add title
ax.set_title("ACF of MTNOY GARCH Model Standardized Residuals");
```



Model Deployment

Ungraded Task: If it's not already running, start your app server.

Task 8.5.18

Tip: How can you access the AIC and BIC scores programmatically? Every `ARCHModelResult` has an `.aic` and a `.bic` attribute.

```
In [21]: def fit(self, p, q):
# Train Model, assign to `self.model`
self.model = arch_model(
    self.data,
    p=p,
    q=q,
    rescale=False,
    vol="GARCH",
    dist="normal",
).fit(disp=0)

# Add AIC and BIC attributes
self.aic = self.model.aic
self.bic = self.model.bic
```

Task 8.5.19

```
In [22]: # Import `FitIn` class and `fit_model` function
from main import FitIn, fit_model

# Instantiate `FitIn` object
request = FitIn(ticker="MTNOY", use_new_data=False, n_observations=2500, p=1, q=

# Build model and fit to data, following parameters in `request`
fit_out = fit_model(request=request)

# Inspect `fit_out`
fit_out
```

```
Out[22]: {'ticker': 'MTNOY',
'use_new_data': False,
'n_observations': 2500,
'p': 1,
'q': 1,
'success': True,
'message': "Trained and saved 'models/2026-07-26T11:22:44.186517_MTNOY.pkl'. Metrics: AIC 12132.023812235264, BIC 12155.31999627869."}
```

Task 8.5.20

```
In [25]: # URL of `/fit` path
url = "http://localhost:8008/fit"

# Data to send to path
json = {
    "ticker": "MTNOY",
    "use_new_data": False,
    "n_observations": 2500,
    "p": 1,
    "q": 1,
}

# Response of post request
response = requests.post(url=url, json=json)

print("response type:", type(response))
print("response status code:", response.status_code)
```

```
response type: <class 'requests.models.Response'>
response status code: 200
```

Task 8.5.21

```
In [27]: # URL of `/predict` path
url = "http://localhost:8008/predict"

# Data to send to path
json = {
    "ticker": "MTNOY",
    "n_days": 5,
}

# Response of post request
response = requests.post(url=url, json=json)

print("response type:", type(response))
print("response status code:", response.status_code)
```

```
response type: <class 'requests.models.Response'>
response status code: 200
```

Copyright 2023 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.